Python প্রোগ্রামিং

সম্পূর্ণ গাইড — Zero থেকে Advanced

Python একদম কখনো দেখোনি? কোনো সমস্যা নেই। এই গাইডটি তোমাকে একদম শুরু থেকে হাত ধরে নিয়ে যাবে — Variables থেকে শুরু করে Object Oriented Programming পর্যন্ত।

🔥 Beginner Friendly | 📖 14 Chapters | 💻 100+ Code Examples | 🌙 Dark Mode | BD বাংলা / GB English

```
>>> print("হ্যালো, Python শিখছি!")
হ্যালো, Python শিখছি!
>>> print("Hello, I'm learning Python!")
Hello, I'm learning Python!
```

সূচিপত্র

01 Python পরিচিতি ও Installation Ch. 1

Python কী, কেন শিখবো, ইনস্টলেশন, প্রথম প্রোগ্রাম

02 print(), input() ও Comments Ch. 2

স্ক্রিনে লেখা, ব্যবহারকারীর কাছ থেকে ইনপুট নেওয়া

03 ভেরিয়েবল ও ডেটা টাইপ Ch. 3

int, float, str, bool, type() — ডেটা সংরক্ষণ

04 অপারেটর Ch. 4

গাণিতিক, তুলনা, লজিক্যাল অপারেটর

05 শর্ত (if / elif / else) Ch. 5

সিদ্ধান্ত নেওয়া — প্রোগ্রামকে চিন্তা করা শেখানো

06 লুপ (for / while) Ch. 6

কাজ বারবার করানো — break, continue, range()

07 ফাংশন Ch. 7

def, return, parameters, *args, **kwargs

08 লিস্ট, টাপল, সেট, ডিকশনারি Ch. 8

Python-এর ৪টি প্রধান ডেটা স্ট্রাকচার

09 স্ট্রিং (String) Ch. 9

টেক্সট নিয়ে কাজ, slicing, methods, f-string

10 ফাইল হ্যান্ডলিং Ch. 10

ফাইল পড়া, লেখা, মুছা — open(), with

11 এরর হ্যান্ডলিং Ch. 11

try, except, finally, raise — ভুল সামলানো

12 OOP — অবজেক্ট অরিয়েন্টেড প্রোগ্রামিং Ch. 12

class, object, __init__, inheritance, polymorphism

13 মডিউল ও লাইব্রেরি Ch. 13

import, os, math, random, datetime — বাইরের কোড ব্যবহার

14 Advanced Python Ch. 14

List Comprehension, Lambda, Decorators, Generators

CHAPTER 01

🐍 Python পরিচিতি ও Installation

Python কী, কেন শিখবো, কিভাবে ইনস্টল করবো এবং প্রথম প্রোগ্রাম

🤔 Python কী?

Python একটি **High-Level Programming Language**। এটি তৈরি করেছেন **Guido van Rossum**, ১৯৯১ সালে। Python-এর সবচেয়ে বড় বৈশিষ্ট্য হলো এর কোড পড়তে অনেকটা ইংরেজির মতো — তাই beginner-দের জন্য এটি সবচেয়ে ভালো।

✅ Python কেন শিখবো?

- শেখা সবচেয়ে সহজ
- Web Development (Django, Flask)
- Data Science & AI/ML
- Automation & Scripting
- Cyber Security / Hacking
- বিশ্বের ১ নম্বর জনপ্রিয় ভাষা
- চাকরির বাজারে সর্বোচ্চ চাহিদা

🔥 Python দিয়ে কী করা যায়?

- 🌐 ওয়েবসাইট ও API
- 🤖 AI / Chatbot
- 📊 ডেটা বিশ্লেষণ
- 🎮 সহজ গেম
- 📱 Desktop App
- 🔐 Hacking টুলস
- ⚙️ অটোমেশন স্ক্রিপ্ট

⚙️ Python ইনস্টল করো

1

Python ডাউনলোড করো

python.org ওয়েবসাইটে যাও → Downloads → তোমার OS অনুযায়ী ডাউনলোড করো (Windows/Mac/Linux)

> ⚠️ Windows-এ install করার সময় অবশ্যই **"Add Python to PATH"** বক্সে টিক দাও!

2

ইনস্টল হয়েছে কিনা চেক করো

Terminal/CMD খুলে টাইপ করো:

```
$ python --version Python 3.12.0 # এরকম কিছু দেখাবে / should show something like this
```

3

Code Editor — VS Code

VS Code ডাউনলোড করো (code.visualstudio.com) — এটাই সবচেয়ে ভালো editor। Python extension ইনস্টল করো।

🎉 প্রথম প্রোগ্রাম লেখো!

VS Code খুলো। একটা নতুন ফাইল তৈরি করো নাম দাও `hello.py` । এবার নিচের কোডটা লেখো:

```
# আমার প্রথম Python প্রোগ্রাম / My first Python program print("হ্যালো দুনিয়া!") print("Hello, World!") print("আমি Python শিখছি!")
```

```
↓ OUTPUT
হ্যালো দুনিয়া! Hello, World! আমি Python শিখছি!
```

Terminal-এ গিয়ে রান করো:

```
$ python hello.py
```

> 🎊 **অভিনন্দন!**
>
> তুমি তোমার প্রথম Python প্রোগ্রাম লিখেছো এবং রান করেছো! এটাই শুরু।

📌 Chapter সারসংক্ষেপ

- 🐍 Python = সহজ, শক্তিশালী, জনপ্রিয় programming language
- ⚙️ python.org থেকে ইনস্টল করো, PATH যোগ করতে ভুলো না
- ✏️ VS Code হলো সবচেয়ে ভালো editor
- ▶️ `python filename.py` দিয়ে প্রোগ্রাম রান করো

CHAPTER 02

🖨️ print(), input() ও Comments

স্ক্রিনে দেখানো, ব্যবহারকারীর কাছ থেকে তথ্য নেওয়া এবং কোডে নোট লেখা

🖨️ print() — স্ক্রিনে দেখানো

`print()` হলো Python-এর সবচেয়ে বেশি ব্যবহৃত function। এটি যা দাও তাই স্ক্রিনে দেখায়।

```
# সাধারণ print / Simple print print("নমস্কার!") print(100) print(3.14) # একসাথে অনেক কিছু print করা / Printing multiple things print("আমার বয়স:", 20) print("My name is", "Rahim", "and I am", 20) # sep দিয়ে separator বদলানো / Change separator with sep print("Python", "Java", "C++", sep=" | ") # end দিয়ে line ending বদলানো / Change line ending with end print("Hello", end=" ") print("World")
```

```
⬇ OUTPUT
নমস্কার! 100 3.14 আমার বয়স: 20 My name is Rahim and I am 20 Python | Java | C++ Hello World
```

⌨️ input() — ব্যবহারকারীর কাছ থেকে তথ্য নেওয়া

`input()` দিয়ে প্রোগ্রাম চলার সময় ব্যবহারকারীর কাছ থেকে তথ্য নেওয়া যায়। **মনে রেখো: input() সব কিছু string হিসেবে নেয়।**

```
# ব্যবহারকারীর নাম নেওয়া / Taking user's name naam = input("তোমার নাম কী? ") print("হ্যালো,", naam) # সংখ্যা নেওয়া — int() দিয়ে convert করতে হবে! # Taking number — must convert with int()! boyos = int(input("তোমার বয়স কত? ")) print("তুমি", boyos, "বছর বয়সী।") # দশমিক সংখ্যা নেওয়া / Taking decimal
```

```
number weight = float(input("তোমার ওজন কত কেজি? ")) print("ওজন:", weight, "kg")
```

⚠️ **গুরুত্বপূর্ণ কথা!**

`input()` সবসময় **string** return করে। গণনার জন্য `int()` বা `float()` দিয়ে convert করতে হবে।

💬 Comments — কোডে নোট লেখা

Comments হলো কোডের মধ্যে মানুষের পড়ার জন্য লেখা নোট। Python এগুলো execute করে না।

```
# এটা একটা single-line comment # This is a single-line comment
print("Hello") # এই লাইনের শেষে comment # Multi-line comment এভাবে লেখো:
""" এটা একটা multi-line comment বা docstring """ print("Comments Python
execute করে না।")
```

💡 **Comment কেন লিখবো?**

ভবিষ্যতে তুমি নিজে বা অন্য কেউ কোড পড়লে যেন বুঝতে পারে। Professional programmer রা সবসময় comment লেখেন।

ছোট্ট প্রজেক্ট: নাম ও বয়স দিয়ে স্বাগতম বার্তা

```
# স্বাগতম প্রোগ্রাম / Welcome program naam = input("তোমার নাম কী? ") boyos =
int(input("তোমার বয়স কত? ")) shahor = input("তুমি কোথায় থাকো? ") print()
# ফাঁকা লাইন / blank line print("────────────────────────") print("
স্বাগতম!") print(" নাম:", naam) print(" বয়স:", boyos, "বছর") print(" শহর:",
shahor) print("────────────────────────")
```

```
↓ OUTPUT
```

```
──────────────────────── স্বাগতম! নাম: Rahim বয়স: 20 বছর শহর: Dhaka
────────────────────────
```

📌 Chapter সারসংক্ষেপ

- 🖨️ `print()` — স্ক্রিনে দেখায়, sep ও end দিয়ে customize করা যায়
- ⌨️ `input()` — সবসময় string দেয়, সংখ্যার জন্য int()/float() লাগে
- 💬 `#` — single line comment, """ """ — multi-line

CHAPTER 03

📦 ভেরিয়েবল ও ডেটা টাইপ

ডেটা সংরক্ষণ করার পাত্র এবং ডেটার ধরন

📦 Variable কী?

Variable হলো একটি **নামযুক্ত বাক্স** যেখানে তুমি ডেটা রাখো। যেমন একটা বাক্সে জিনিস রাখলে বাক্সের নাম ধরে সেটা খোঁজা যায়।

🧠 ধারণাটা বোঝো

```
naam  = "Rahim"    ← "naam" নামের বাক্সে "Rahim" রাখলাম
boyos = 20         ← "boyos" নামের বাক্সে 20 রাখলাম
GPA   = 4.75       ← "GPA" বাক্সে 4.75 রাখলাম
```

```
# Variable তৈরি করা / Creating variables naam = "Rahim" # String (লেখা/text) boyos = 20 # Integer (পূর্ণ সংখ্যা) ojon = 65.5 # Float (দশমিক সংখ্যা) student = True # Boolean (সত্য/মিথ্যা) # Variable ব্যবহার করা / Using variables print(naam) print(boyos) print(ojon) print(student)
```

```
↓ OUTPUT
Rahim 20 65.5 True
```

📁 Python-এর প্রধান ডেটা টাইপ

Data Type	বাংলায়	উদাহরণ	ব্যবহার
int	পূর্ণ সংখ্যা	20, -5, 1000	বয়স, সংখ্যা গোনা
float	দশমিক সংখ্যা	3.14, -2.5	ওজন, তাপমাত্রা
str	লেখা/পাঠ্য	"Rahim", 'Hello'	নাম, বার্তা
bool	সত্য বা মিথ্যা	True, False	শর্ত পরীক্ষা
list	পরিবর্তনযোগ্য তালিকা	[1, 2, 3]	একাধিক আইটেম
dict	key-value জোড়া	{"naam": "Rahim"}	ডেটা mapping
NoneType	কিছু নেই / খালি	None	শূন্য মান

🔍 type() — ডেটা টাইপ জানো

```
# type() দিয়ে ডেটা টাইপ চেক করো print(type(20)) # <class 'int'>
print(type(3.14)) # <class 'float'> print(type("Hello")) # <class 'str'>
print(type(True)) # <class 'bool'> print(type(None)) # <class 'NoneType'>
```

```
↓ OUTPUT
<class 'int'> <class 'float'> <class 'str'> <class 'bool'> <class
'NoneType'>
```

🔄 Type Conversion — এক টাইপ থেকে অন্যটায়

```
# String থেকে Integer / String to Integer katha = "50" sankhya =
int(katha) # "50" → 50 print(sankhya + 10) # 60 # Integer থেকে Float /
Integer to Float a = 5 b = float(a) # 5 → 5.0 print(b) # যেকোনো কিছু থেকে
String / Any to String boyos = 20 text = str(boyos) # 20 → "20"
print("বয়স: " + text) # string জোড়া লাগানো
```

```
⬇ OUTPUT
60 5.0 বয়স: 20
```

📏 Variable-এর নামকরণের নিয়ম

✅ সঠিক নাম

```
naam = "Rahim" my_name =
"Karim" age2 = 25 _private =
100 myGPA = 4.75
```

❌ ভুল নাম

```
2naam = "Rahim" # সংখ্যা দিয়ে
শুরু না my-name = "Karim" #
dash নেই for = 10 # keyword
নয় my name = 100 # space নেই
```

📌 Chapter সারসংক্ষেপ

- 📦 Variable = ডেটা রাখার নামযুক্ত বাক্স
- 📊 প্রধান টাইপ: int, float, str, bool, None
- 🔍 `type()` — টাইপ জানতে
- 🔄 `int()`, `float()`, `str()` — টাইপ বদলাতে

CHAPTER 04

🔢 অপারেটর

গাণিতিক হিসাব, তুলনা এবং লজিক — Python কিভাবে চিন্তা করে

➕ গাণিতিক অপারেটর

Operator	কাজ	উদাহরণ	Result
+	যোগ	10 + 3	13
-	বিয়োগ	10 - 3	7
*	গুণ	10 * 3	30
/	ভাগ (দশমিক)	10 / 3	3.333...
//	পূর্ণ ভাগ	10 // 3	3
%	ভাগশেষ (modulo)	10 % 3	1
**	পাওয়ার/ঘাত	2 ** 10	1024

```
# Calculator উদাহরণ / Calculator example a = 20 b = 6 print("যোগ:", a +
b) # 26 print("বিয়োগ:", a - b) # 14 print("গুণ:", a * b) # 120
print("ভাগ:", a / b) # 3.333... print("পূর্ণ ভাগ:", a // b) # 3
print("ভাগশেষ:", a % b) # 2 print("2 এর 10 ঘাত:", 2 ** 10) # 1024
```

🤔 তুলনা অপারেটর (Comparison)

এগুলো দুটো মান তুলনা করে এবং True বা False দেয়।

Operator	অর্থ	উদাহরণ	Result
==	সমান কিনা	5 == 5	True
≠	সমান নয়	5 ≠ 3	True
>	বড়	10 > 5	True
<	ছোট	3 < 7	True
≥	বড় বা সমান	5 ≥ 5	True
≤	ছোট বা সমান	4 ≤ 5	True

🧠 লজিক্যাল অপারেটর

Operator	অর্থ	উদাহরণ	Result
and	দুটোই সত্য হলে True	True and True	True
or	যেকোনো একটা সত্য হলে True	True or False	True
not	উল্টো করে দেয়	not True	False

```
# Logical operators উদাহরণ boyos = 20 student = True print(boyos ≥ 18 and student) # True and True = True print(boyos < 18 or student) # False or True = True print(not student) # not True = False
```

```
↓ OUTPUT
True True False
```

🔧 Assignment Operators (শর্টকাট)

```
x = 10 x += 5 # x = x + 5 → x = 15 x -= 3 # x = x - 3 → x = 12 x *= 2 # x = x * 2 → x = 24 x //= 4 # x = x // 4 → x = 6 x **= 2 # x = x ** 2 → x = 36 print(x) # 36
```

📌 Chapter সারসংক্ষেপ

- ➕ Arithmetic: +, -, *, /, //, %, ** — গাণিতিক কাজ
- 🤔 Comparison: ==, !=, >, <, >=, <= — True/False দেয়
- 🧠 Logical: and, or, not — শর্ত মিলানো
- 🔧 Assignment: +=, -=, *= — শর্টকাট

CHAPTER 05

🚦 শর্ত — if / elif / else

প্রোগ্রামকে সিদ্ধান্ত নেওয়া শেখানো — বিভিন্ন পরিস্থিতিতে বিভিন্ন কাজ

🚦 if — শর্ত পরীক্ষা

বাস্তব জীবনে আমরা প্রতিনিয়ত সিদ্ধান্ত নিই — "যদি বৃষ্টি হয়, তাহলে ছাতা নেব।" Python-এও একই ভাবে কাজ করে।

🧠 গঠন / Structure

```
if শর্ত:
    কাজ (যদি শর্ত সত্য হয়)
elif অন্য শর্ত:
    কাজ (যদি দ্বিতীয় শর্ত সত্য)
else:
    কাজ (যদি কোনো শর্ত না মেলে)
```

⚠️ Indentation — বাংলায় বলতে গেলে "ভেতর ঢোকানো"

Python-এ if-এর পরের কোড **4টা space বা 1টা Tab** দিয়ে ভেতরে লিখতে হয়। এটা না করলে error আসবে!

```
# সহজ if উদাহরণ / Simple if example boyos = int(input("তোমার বয়স কত? ")) if boyos ≥ 18: print("তুমি প্রাপ্তবয়স্ক! ভোট দিতে পারবে।") elif boyos ≥ 13: print("তুমি কিশোর। আর কিছুদিন অপেক্ষা করো।") else: print("তুমি শিশু। স্কুলে মনোযোগ দাও!")
```

Grade Calculator উদাহরণ

```
marks = int(input("নম্বর দাও (0-100): ")) if marks ≥ 80: grade = "A+" comment = "অসাধারণ! 🎉" elif marks ≥ 70: grade = "A" comment = "খুব ভালো! 👍" elif marks ≥ 60: grade = "B" comment = "ভালো করেছো।" elif marks ≥ 50: grade = "C" comment = "আরো চেষ্টা করো।" elif marks ≥ 33: grade = "D" comment = "পাস হয়েছো।" else: grade = "F" comment = "ফেল। আবার চেষ্টা করো! 💪" print(f"গ্রেড: {grade} — {comment}")
```

Nested if — if-এর ভেতরে if

```
boyos = 20 citizen = True if boyos ≥ 18: if citizen: print("ভোট দিতে পারবে ✅") else: print("বয়স হয়েছে কিন্তু নাগরিক নও ❌") else: print("বয়স হয়নি ❌")
```

```
⬇ OUTPUT
ভোট দিতে পারবে ✅
```

📌 Chapter সারসংক্ষেপ

- 🚦 `if` — একটি শর্ত পরীক্ষা
- 🔀 `elif` — আরেকটি শর্ত (একাধিক হতে পারে)
- 🔚 `else` — কোনো শর্ত না মিললে এটা চলে
- 📐 Indentation — 8 space বা Tab, এটা Python-এ বাধ্যতামূলক!

CHAPTER 06

🔄 লুপ — for / while

কাজ বারবার করানো — Python-এর সবচেয়ে শক্তিশালী বৈশিষ্ট্যগুলোর একটি

🔁 for loop — নির্দিষ্ট সংখ্যকবার

`for` loop ব্যবহার করা হয় যখন জানি কতবার কাজ করতে হবে, অথবা একটি list/sequence-এর সব item-এর উপর কাজ করতে হবে।

```
# 5 বার Hello প্রিন্ট করো / Print Hello 5 times for i in range(5):
print("Hello!", i) print("--- range(1, 6) ---") # 1 থেকে 5 পর্যন্ত / 1 to 5
for i in range(1, 6): print(i, end=" ")
```

```
⬇ OUTPUT
Hello! 0 Hello! 1 Hello! 2 Hello! 3 Hello! 4 --- range(1, 6) --- 1 2 3 4
5
```

```
# List-এর উপর for loop / for loop over a list fruits = ["আম", "কলা",
"লিচু", "আনারস"] for fol in fruits: print("ফল:", fol) # String-এর উপর
loop / loop over a string for akhor in "Python": print(akhor, end="-")
```

```
⬇ OUTPUT
ফল: আম ফল: কলা ফল: লিচু ফল: আনারস P-y-t-h-o-n-
```

🔂 while loop — শর্ত সত্য থাকলে চলবে

```
# 1 থেকে শুরু করে 5 পর্যন্ত / Count from 1 to 5 sankhya = 1 while sankhya ≤ 5: print(sankhya) sankhya += 1 # এটা না দিলে infinite loop হবে! print("শেষ!")
```

```
↓ OUTPUT
1 2 3 4 5 শেষ!
```

⛔ break ও 🔀 continue

```
# break — loop থামিয়ে দেয় / stops the loop for i in range(1, 11): if i == 5: break # 5 আসলে থামো print(i, end=" ") print() # continue — এই iteration skip করো / skip this iteration for i in range(1, 11): if i % 2 == 0: continue # জোড় সংখ্যা skip করো print(i, end=" ")
```

```
↓ OUTPUT
1 2 3 4 1 3 5 7 9
```

প্র্যাক্টিক্যাল উদাহরণ: গুণের নামতা

```
sankhya = int(input("কোন সংখ্যার নামতা দেখাবো? ")) print(f"\n{sankhya}-এর নামতা:") print("-" * 20) for i in range(1, 11): print(f"{sankhya} × {i:2d} = {sankhya * i}")
```

```
↓ OUTPUT (যদি 5 দাও)
5-এর নামতা: ──────────────────── 5 × 1 = 5 5 × 2 = 10 5 × 3 = 15 ... 5 × 10 = 50
```

📌 Chapter সারসংক্ষেপ

- 🔁 `for` — নির্দিষ্ট বার বা list-এর প্রতিটি item-এ কাজ

- 🔁 `while` — শর্ত সত্য থাকলে চলে
- 📊 `range(start, stop, step)` — সংখ্যার ক্রম তৈরি
- ⛔ `break` — loop থামায়, `continue` — এই iteration skip করে

CHAPTER 07

⚙️ ফাংশন

কোড পুনরায় ব্যবহারযোগ্য টুকরোতে ভাগ করা — def, return, parameters

⚙️ Function কী এবং কেন?

Function হলো কোডের একটা **নামযুক্ত block** যা একটি নির্দিষ্ট কাজ করে। একবার লিখলে বারবার ব্যবহার করা যায়। রান্নার recipe-র মতো — একবার লিখলে যখন খুশি ব্যবহার করো।

```
# Function তৈরি করা / Creating a function def shobaike_salam():
print("আস্‌সালামু আলাইকুম!") print("সবাইকে স্বাগতম!") # Function call করা /
Calling the function shobaike_salam() # প্রথমবার shobaike_salam() # দ্বিতীয়বার
shobaike_salam() # তৃতীয়বার
```

```
↓ OUTPUT
আস্‌সালামু আলাইকুম! সবাইকে স্বাগতম! আস্‌সালামু আলাইকুম! সবাইকে স্বাগতম! আস্‌সালামু
আলাইকুম! সবাইকে স্বাগতম!
```

📥 Parameters — তথ্য পাঠানো

```
# Parameter সহ function / Function with parameters def salam_dao(naam):
print(f"হ্যালো, {naam}! কেমন আছো?") salam_dao("Rahim") salam_dao("Karim")
# একাধিক parameter / Multiple parameters def yog_koro(a, b): fol = a + b
print(f"{a} + {b} = {fol}") yog_koro(10, 20) yog_koro(5, 8)
```

```
↓ OUTPUT
হ্যালো, Rahim! কেমন আছো? হ্যালো, Karim! কেমন আছো? 10 + 20 = 30 5 + 8 = 13
```

📤 return — ফলাফল ফেরত পাঠানো

```
# return দিয়ে মান ফেরত দেওয়া def chorbritto(bhumi, uchho): return (bhumi *
uchho) / 2 # return করা মান ব্যবহার করা fol = chorbritto(10, 6)
print(f"ত্রিভুজের ক্ষেত্রফল: {fol}") # সরাসরিও ব্যবহার করা যায় print(f"ক্ষেত্রফল:
{chorbritto(8, 4)}")
```

```
↓ OUTPUT
ত্রিভুজের ক্ষেত্রফল: 30.0 ক্ষেত্রফল: 16.0
```

📌 Default Parameter

```
# Default value দেওয়া / Setting default values def parichoy(naam,
bhoomi="বাংলাদেশ"): print(f"নাম: {naam}, দেশ: {bhoomi}") parichoy("Rahim")
# default দেশ parichoy("John", "USA") # custom দেশ parichoy("Priya",
bhoomi="India") # keyword argument
```

```
↓ OUTPUT
নাম: Rahim, দেশ: বাংলাদেশ নাম: John, দেশ: USA নাম: Priya, দেশ: India
```

🎯 *args ও **kwargs — যেকোনো সংখ্যক argument

```
# *args — যেকোনো সংখ্যক positional argument def sob_jog_koro(*sankhya):
total = sum(sankhya) return total print(sob_jog_koro(1, 2, 3)) # 6
print(sob_jog_koro(10, 20, 30, 40)) # 100 # **kwargs — যেকোনো keyword
arguments def profile_dakaho(**tothyo): for chabi, mon in tothyo.items():
print(f" {chabi}: {mon}") profile_dakaho(naam="Rahim", boyos=20,
shahor="Dhaka")
```

```
↓ OUTPUT
6 100 naam: Rahim boyos: 20 shahor: Dhaka
```

📌 Chapter সারসংক্ষেপ

- ⚙️ `def naam():` — function তৈরি
- 📥 Parameters — function-এ তথ্য পাঠাও
- 📤 `return` — ফলাফল ফেরত দাও
- 📌 Default parameter — না দিলে এটা ব্যবহার হয়
- 🎯 `*args, **kwargs` — যতো খুশি argument

CHAPTER 08

📚 List, Tuple, Set, Dictionary

Python-এর ৪টি প্রধান ডেটা স্ট্রাকচার — একসাথে অনেক ডেটা রাখো

📋 List — পরিবর্তনযোগ্য তালিকা

List হলো একটি ordered (ক্রমানুযায়ী) এবং changeable (পরিবর্তনযোগ্য) collection। Square brackets `[ ]` দিয়ে তৈরি হয়।

```
# List তৈরি করা / Creating a list fruits = ["আম", "কলা", "লিচু", "আনারস"]
sankhya = [1, 2, 3, 4, 5] mishrana = [1, "Rahim", 3.14, True] # mixed
types # Index দিয়ে access / Access by index (0 থেকে শুরু!) print(fruits[0])
# আম (প্রথম) print(fruits[2]) # লিচু print(fruits[-1]) # আনারস (শেষ) #
List পরিবর্তন / Modify list fruits[1] = "পেয়ারা" print(fruits) # ['আম',
'পেয়ারা', 'লিচু', 'আনারস'] # নতুন item যোগ / Add items
fruits.append("কমলা") # শেষে যোগ fruits.insert(1, "জাম") # index 1-এ যোগ
# item সরানো / Remove items fruits.remove("লিচু") # নির্দিষ্ট item fruits.pop()
# শেষেরটা সরাও fruits.pop(0) # index 0 সরাও # দরকারি method / Useful
methods print(len(fruits)) # দৈর্ঘ্য print(sorted(sankhya)) # সাজানো
print(sum(sankhya)) # যোগফল print(max(sankhya)) # সর্বোচ্চ
print(min(sankhya)) # সর্বনিম্ন
```

Slicing — List-এর অংশ বের করা

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] print(nums[2:5]) # [2, 3, 4] —
index 2 থেকে 4 print(nums[:4]) # [0, 1, 2, 3] — শুরু থেকে print(nums[6:])
# [6, 7, 8, 9] — শেষ পর্যন্ত print(nums[::2]) # [0, 2, 4, 6, 8] — একটা করে
skip print(nums[::-1]) # [9, 8, 7, ...] — উল্টো
```

🔒 Tuple — পরিবর্তন-অযোগ্য তালিকা

Tuple, List-এর মতোই কিন্তু এটা **পরিবর্তন করা যায় না**। Parentheses `( )` দিয়ে তৈরি।

```
desh = ("Bangladesh", "India", "Pakistan") coordinates = (23.8103, 90.4125) # Dhaka-র lat, lon print(desh[0]) # Bangladesh print(len(desh)) # 3 # Tuple unpack করা / Unpacking lat, lon = coordinates print(f"Latitude: {lat}, Longitude: {lon}")
```

🎲 Set — অনন্য মানের সংগ্রহ

Set-এ duplicate হয় না। Curly braces `{ }` দিয়ে তৈরি।

```
fruits = {"আম", "কলা", "আম", "লিচু", "কলা"} print(fruits) # {'আম', 'কলা', 'লিচু'} — duplicate নেই! # Set operations a = {1, 2, 3, 4} b = {3, 4, 5, 6} print(a | b) # Union: {1,2,3,4,5,6} print(a & b) # Intersection: {3,4} print(a - b) # Difference: {1,2}
```

📁 Dictionary — Key-Value জোড়া

Dictionary হলো key-value pair-এর সংগ্রহ। একটা আসল Dictionary-র মতো — word (key) দিয়ে অর্থ (value) খোঁজো।

```
# Dictionary তৈরি / Create student = { "naam": "Rahim", "boyos": 20, "GPA": 4.75, "subject": ["Math", "Physics"] } # Access / পড়া print(student["naam"]) # Rahim print(student.get("GPA")) # 4.75 (safer) # Add / Update student["shahor"] = "Dhaka" # নতুন key student["boyos"] = 21 # update # Loop দিয়ে সব দেখাও for chabi, mon in student.items(): print(f"{chabi}: {mon}") # Keys ও Values আলাদা print(student.keys()) print(student.values())
```

🔑 কোনটা কখন ব্যবহার করবো?

Type	ব্যবহারের সময়
`list`	ক্রমানুযায়ী, পরিবর্তনযোগ্য তালিকা
`tuple`	স্থির ডেটা (coordinates, settings)
`set`	unique মান, duplicate সরাতে
`dict`	label দিয়ে ডেটা খোঁজা

📌 Chapter সারসংক্ষেপ

- 📋 `list [ ]` — ordered, changeable — সবচেয়ে বেশি ব্যবহৃত
- 🔒 `tuple ( )` — ordered, unchangeable
- 🎲 `set { }` — unordered, duplicate নেই
- 📁 `dict { key: value }` — key দিয়ে value খোঁজো

CHAPTER 09

📝 স্ট্রিং (String)

টেক্সট নিয়ে কাজ — Python-এ সবচেয়ে বেশি ব্যবহৃত ডেটা টাইপ

📝 String তৈরি করার উপায়

```
s1 = 'Single quote' s2 = "Double quote" s3 = """তিন লাইনের স্ট্রিং""" # String কোড বা special character s4 = "নতুন লাইন:\nএটা নতুন লাইনে" s5 = "Tab:\tএটা tab পরে" print(s4) print(s5)
```

✂️ String Slicing — অংশ বের করা

```
text = "Python Programming" # 0123456789... print(text[0]) # P print(text[-1]) # g (শেষ অক্ষর) print(text[0:6]) # Python print(text[7:]) # Programming print(text[::-1]) # gnimmargorP nohtyP (উল্টো)
```

🔧 দরকারি String Methods

```
text = " Hello, World! " print(text.upper()) # HELLO, WORLD! print(text.lower()) # hello, world! print(text.strip()) # "Hello, World!" (দুপাশের space সরায়) print(text.replace("World", "Python")) # Hello, Python! print(text.find("World")) # 8 (index) print(text.count("l")) # 3 print(text.startswith("Hello")) # False (space আছে) print(text.strip().startswith("Hello")) # True # Split ও Join words = "Python,Java,C++,JavaScript" list_words = words.split(",")
```

```
print(list_words) # ['Python', 'Java', 'C++', 'JavaScript'] joined = " | ".join(list_words) print(joined) # Python | Java | C++ | JavaScript
```

🎨 f-String — সুন্দরভাবে ডেটা দেখানো

f-string হলো সবচেয়ে আধুনিক এবং সহজ উপায় string-এ variable ঢোকানোর।

```
naam = "Rahim" boyos = 20 GPA = 4.75 # পুরনো পদ্ধতি (এড়িয়ে চলো) / Old way (avoid) print("নাম: " + naam + ", বয়স: " + str(boyos)) # আধুনিক f-string (সবসময় এটা ব্যবহার করো) print(f"নাম: {naam}, বয়স: {boyos}, GPA: {GPA}") # Expression ব্যবহার করা যায় print(f"10 বছর পর বয়স হবে: {boyos + 10}") # Formatting dam = 1234567.89 print(f"দাম: {dam:,.2f} টাকা") # 1,234,567.89 print(f"GPA: {GPA:.1f}") # 4.8
```

```
⬇ OUTPUT
নাম: Rahim, বয়স: 20, GPA: 4.75 10 বছর পর বয়স হবে: 30 দাম: 1,234,567.89 টাকা GPA: 4.8
```

📌 Chapter সারসংক্ষেপ

- ✂️ Slicing: text[start:end:step]
- 🔧 Methods: upper(), lower(), strip(), replace(), split(), join()
- 🎨 `f"{variable}"` — সবচেয়ে সহজ string formatting

CHAPTER 10

📂 ফাইল হ্যান্ডলিং

ফাইল পড়া, লেখা এবং সংরক্ষণ — ডেটা permanently রাখা

📂 open() — ফাইল খোলা

Mode	কাজ	ব্যাখ্যা
"r"	পড়া	শুধু পড়া যাবে, ফাইল থাকতে হবে
"w"	লেখা	নতুন লেখা, আগের সব মুছে যাবে
"a"	যোগ করা	শেষে যোগ হবে, আগেরটা থাকবে
"r+"	পড়া ও লেখা	দুটোই করা যাবে

✍️ ফাইলে লেখা

```
# with statement ব্যবহার করো — automatically close হয় with
open("students.txt", "w", encoding="utf-8") as file:
file.write("Rahim\n") file.write("Karim\n") file.write("Salam\n")
print("ফাইলে লেখা হয়েছে!") # List থেকে লেখা / Write from list students =
["Rahim\n", "Karim\n", "Salam\n"] with open("students.txt", "w",
encoding="utf-8") as file: file.writelines(students)
```

📖 ফাইল পড়া

```
# সব একসাথে পড়া / Read all at once with open("students.txt", "r",
encoding="utf-8") as file: content = file.read() print(content) # লাইন বাই
লাইন পড়া / Read line by line with open("students.txt", "r",
encoding="utf-8") as file: for line in file: print(line.strip()) #
strip() দিয়ে \n সরাও # সব লাইন list-এ / All lines in a list with
open("students.txt", "r", encoding="utf-8") as file: lines =
file.readlines() print(lines) # ['Rahim\n', 'Karim\n', 'Salam\n']
```

➕ ফাইলে যোগ করা (Append)

```
# "a" mode — আগেরটা না মুছে যোগ করা with open("students.txt", "a",
encoding="utf-8") as file: file.write("নতুন ছাত্র\n") print("যোগ হয়েছে!")
```

💡 `with` **কেন ব্যবহার করবো?** `with` ?

`with` ব্যবহার করলে কাজ শেষে ফাইল automatically বন্ধ হয়ে যায়। এটা সবসময় ব্যবহার করা উচিত।

📌 Chapter সারসংক্ষেপ

- 📂 `open(file, mode, encoding)` — ফাইল খোলা
- ✍️ `"w"` — লেখা (মুছে), `"a"` — যোগ করা
- 📖 `read()`, `readlines()` — ফাইল পড়া
- 🔑 `with open() as file:` — সবসময় এভাবে খোলো

CHAPTER 11

🛡️ এরর হ্যান্ডলিং

প্রোগ্রাম crash না করে ভুল সামলানো — try, except, finally

❌ Error বা Exception কী?

Python-এ কোনো ভুল হলে program crash করে। কিন্তু আমরা চাই program crash না করে error সামলাক। এর জন্য `try-except` ব্যবহার করি।

```
# এরকম করলে crash হবে / This will crash # x = int("abc") ← ValueError! # y = 10 / 0 ← ZeroDivisionError! # try-except দিয়ে সামলাও / Handle with try-except try: sankhya = int(input("একটা সংখ্যা দাও: ")) fol = 100 / sankhya print(f"100 ÷ {sankhya} = {fol}") except ValueError: print("❌ সংখ্যা দাও, অক্ষর নয়!") except ZeroDivisionError: print("❌ শূন্য দিয়ে ভাগ দেওয়া যাবে না!") else: print("✅ সফলভাবে হিসাব হয়েছে!") finally: print("এটা সবসময় চলে!")
```

📋 সাধারণ Exception-গুলো

Exception	কখন হয়	উদাহরণ
`ValueError`	ভুল মান	`int("abc")`
`ZeroDivisionError`	শূন্য দিয়ে ভাগ	`10/0`
`IndexError`	List-এ নেই এমন index	`list[100]`
`KeyError`	Dict-এ নেই এমন key	`dict["x"]`

Exception	কখন হয়	উদাহরণ
`TypeError`	ভুল টাইপে অপারেশন	`"a" + 1`
`FileNotFoundError`	ফাইল নেই	`open("x.txt")`
`NameError`	Variable নেই	`print(x)`

🚀 raise — নিজেই Error তৈরি করা

```
def boyos_check(boyos): if boyos < 0: raise ValueError("বয়স নেগেটিভ হতে পারে না!") if boyos > 150: raise ValueError("বয়স ১৫০-এর বেশি হওয়া অসম্ভব!") return f"বয়স {boyos} বছর — বৈধ!" try: print(boyos_check(-5)) except ValueError as e: print(f"Error: {e}")
```

```
↓ OUTPUT
Error: বয়স নেগেটিভ হতে পারে না!
```

📌 Chapter সারসংক্ষেপ

- 🛡️ `try` — ঝুঁকিপূর্ণ কোড এখানে
- ❌ `except` — error হলে এটা চলে
- ✅ `else` — কোনো error না হলে
- 🔚 `finally` — সবসময় চলে (cleanup)
- 🚀 `raise` — নিজে error তৈরি করো

CHAPTER 12

🏗️ OOP — অবজেক্ট অরিয়েন্টেড প্রোগ্রামিং

বাস্তব জগতের মতো করে প্রোগ্রামিং — Class, Object, Inheritance

🤔 OOP কেন দরকার?

ধরো তুমি একটা School management system বানাচ্ছো। অনেক ছাত্র আছে, প্রত্যেকের নাম, বয়স, GPA আলাদা। OOP দিয়ে একটা "Student" নামের ছাঁচ (template) বানাও, তারপর সেখান থেকে যত খুশি ছাত্র তৈরি করো।

🧠 Class vs Object

```
Class = ছাঁচ/Blueprint  (যেমন: গাড়ির ডিজাইন)
Object = আসল জিনিস     (যেমন: সেই ডিজাইন থেকে বানানো গাড়ি)

Student (Class)
├── naam: str
├── boyos: int
├── GPA: float
└── parichoy(): method

rahim = Student("Rahim", 20, 4.75)   ← Object তৈরি
karim = Student("Karim", 21, 3.90)   ← আরেকটি Object
```

🏗️ Class তৈরি করা

```
# Class তৈরি / Create a class class Student: # __init__ — Object তৈরির সময় চলে def __init__(self, naam, boyos, GPA): self.naam = naam # instance variable self.boyos = boyos self.GPA = GPA # Method — object-এর কাজ def parichoy(self): print(f"নাম: {self.naam}") print(f"বয়স: {self.boyos}") print(f"GPA: {self.GPA}") def scholarship_eligible(self): if self.GPA ≥ 4.5: return f"{self.naam} বৃত্তির যোগ্য! ✅" else: return f"{self.naam} বৃত্তির যোগ্য নয়।" # Object তৈরি / Create objects rahim = Student("Rahim", 20, 4.75) karim = Student("Karim", 21, 3.90) # Method call করা / Calling methods rahim.parichoy() print(rahim.scholarship_eligible()) print(karim.scholarship_eligible())
```

```
↓ OUTPUT
নাম: Rahim বয়স: 20 GPA: 4.75 Rahim বৃত্তির যোগ্য! ✅ Karim বৃত্তির যোগ্য নয়।
```

🧬 Inheritance — উত্তরাধিকার

একটা class-এর সব কিছু আরেকটা class-এ পাওয়া। বাবার বৈশিষ্ট্য ছেলের মধ্যে আসার মতো।

```
# Parent class class Manush: def __init__(self, naam, boyos): self.naam = naam self.boyos = boyos def kotha_bolo(self): print(f"{self.naam} কথা বলতে পারে।") # Child class — Manush থেকে inherit করে class Student(Manush): def __init__(self, naam, boyos, GPA): super().__init__(naam, boyos) # parent-এর init self.GPA = GPA def poro(self): print(f"{self.naam} পড়াশোনা করে।") # Teacher class class Teacher(Manush): def porao(self): print(f"{self.naam} পড়ান।") s = Student("Rahim", 20, 4.75) s.kotha_bolo() # Parent-এর method s.poro() # নিজের method
```

```
↓ OUTPUT
Rahim কথা বলতে পারে। Rahim পড়াশোনা করে।
```

📌 Chapter সারসংক্ষেপ

- 🏗️ `class` — Blueprint/Template তৈরি
- 🔑 `__init__` — Object তৈরির সময় চলে (constructor)
- 📌 `self` — নিজের object-কে বোঝায়
- 🧬 `class Child(Parent)` — Inheritance
- 🔁 `super()` — Parent class-এর method ডাকে

CHAPTER 13

📦 মডিউল ও লাইব্রেরি

অন্যদের লেখা কোড ব্যবহার করা — import, os, math, random, datetime

📦 Module কী?

Module হলো Python কোডের একটি ফাইল যা অন্য প্রোগ্রামে `import` করে ব্যবহার করা যায়। Python-এ হাজার হাজার built-in এবং third-party module আছে।

```
# import করার উপায় / Ways to import import math # পুরো module import random as rnd # alias দিয়ে from datetime import datetime # নির্দিষ্ট part from os import * # সব (সাধারণত avoid করো)
```

🔢 math — গণিতের module

```
import math print(math.sqrt(16)) # 4.0 — বর্গমূল print(math.pow(2, 10)) # 1024.0 — ঘাত print(math.pi) # 3.14159... — পাই print(math.ceil(4.2)) # 5 — উপরে গোলাকার print(math.floor(4.9)) # 4 — নিচে গোলাকার print(math.factorial(5)) # 120 — factorial print(math.log(100, 10)) # 2.0 — log base 10
```

🎲 random — এলোমেলো সংখ্যা

```
import random print(random.randint(1, 100)) # 1-100 এর মধ্যে যেকোনো print(random.random()) # 0.0 - 1.0 print(random.choice(["আম", "কলা",
```

```
"লিচু"])) # list থেকে এলোমেলো fruits = ["আম", "কলা", "লিচু", "আনারস"]
random.shuffle(fruits) # list shuffle করো print(fruits)
```

📅 datetime — তারিখ ও সময়

```
from datetime import datetime, date, timedelta # এখনকার সময় / Current
time ekhon = datetime.now() print(ekhon) # 2025-01-15 14:30:00
print(ekhon.strftime("%d/%m/%Y")) # 15/01/2025 print(ekhon.year,
ekhon.month, ekhon.day) # তারিখের হিসাব / Date calculation janmo =
date(2005, 6, 15) aaj = date.today() boyos = aaj.year - janmo.year
print(f"বর্তমান বয়স: {boyos} বছর") # ১০০ দিন পরের তারিখ porerdin = aaj +
timedelta(days=100) print(f"১০০ দিন পরে: {porerdin}")
```

💻 os — অপারেটিং সিস্টেম

```
import os print(os.getcwd()) # বর্তমান folder print(os.listdir(".")) #
folder-এর ফাইল os.mkdir("new_folder") # নতুন folder
print(os.path.exists("file.txt")) # আছে কিনা print(os.path.join("folder",
"file.txt")) # path join
```

📦 pip — Third-Party Library ইনস্টল করা

```
# Terminal-এ চালাও / Run in terminal $ pip install requests # ওয়েব request
$ pip install numpy # সংখ্যার হিসাব $ pip install pandas # ডেটা বিশ্লেষণ $ pip
install matplotlib # graph/chart $ pip install flask # ওয়েব সার্ভার
```

📌 Chapter সারসংক্ষেপ

- 📦 `import module` — module ব্যবহার করো

- `math` — গণিতের কাজ
- `random` — এলোমেলো ডেটা
- `datetime` — তারিখ ও সময়
- `os` — ফাইল সিস্টেম
- `pip install` — বাইরের library ইনস্টল

CHAPTER 14

🚀 Advanced Python

List Comprehension, Lambda, Decorators, Generators — Pro-level Python

⚡ List Comprehension — একলাইনে list তৈরি

সাধারণ loop-এর বদলে একটি compact way-তে list তৈরি করার পদ্ধতি।

```
# সাধারণ পদ্ধতি / Regular way squares = [] for i in range(1, 6):
squares.append(i ** 2) print(squares) # [1, 4, 9, 16, 25] # List
Comprehension — এক লাইনে! squares = [i ** 2 for i in range(1, 6)]
print(squares) # [1, 4, 9, 16, 25] # শর্ত সহ / With condition jugolr = [i
for i in range(1, 21) if i % 2 == 0] print(jugolr) # [2, 4, 6, 8, 10, 12,
14, 16, 18, 20] # String list থেকে / From string list fruits = ["apple",
"banana", "mango"] upper_fruits = [f.upper() for f in fruits]
print(upper_fruits) # ['APPLE', 'BANANA', 'MANGO']
```

λ Lambda — নামছাড়া ছোট Function

```
# সাধারণ function / Regular function def dwigun(x): return x * 2 # Lambda
দিয়ে / Using lambda dwigun = lambda x: x * 2 print(dwigun(5)) # 10 #
একাধিক parameter / Multiple parameters jog = lambda a, b: a + b
print(jog(10, 20)) # 30 # sorted() এর সাথে / With sorted() students =
[("Rahim", 4.75), ("Karim", 3.90), ("Salam", 4.50)] sorted_students =
sorted(students, key=lambda x: x[1], reverse=True) print(sorted_students)
# GPA অনুযায়ী সাজানো
```

🎁 Decorators — Function-এ extra শক্তি যোগ

```
# Decorator তৈরি / Creating a decorator def uppercase_decorator(func): def wrapper(*args, **kwargs): result = func(*args, **kwargs) return result.upper() return wrapper # @ দিয়ে apply করা / Applying with @ @uppercase_decorator def greet(naam): return f"hello, {naam}!" print(greet("rahim")) # HELLO, RAHIM! # Timing decorator — কতক্ষণ লাগলো import time def timer(func): def wrapper(*args, **kwargs): start = time.time() result = func(*args, **kwargs) end = time.time() print(f"{func.__name__} সময় নিলো: {end-start:.4f}s") return result return wrapper @timer def gorono(): total = sum(range(1000000)) return total gorono()
```

⚡ Generators — মেমরি-দক্ষ Iteration

```
# Generator function — yield ব্যবহার করে def count_up(n): i = 1 while i ≤ n: yield i # return-এর মতো, কিন্তু থামে না i += 1 # ব্যবহার করা / Using it for num in count_up(5): print(num, end=" ") # 1 2 3 4 5 # Generator Expression (list comprehension-এর মতো) gen = (i ** 2 for i in range(10)) print(next(gen)) # 0 print(next(gen)) # 1 print(next(gen)) # 4
```

🗺️ map() ও filter()

```
# map() — প্রতিটি item-এ function apply করো numbers = [1, 2, 3, 4, 5] doubled = list(map(lambda x: x * 2, numbers)) print(doubled) # [2, 4, 6, 8, 10] # filter() — শর্ত মেলে এমনগুলো রাখো jugol = list(filter(lambda x: x % 2 == 0, numbers)) print(jugol) # [2, 4] # reduce() — সব মিলিয়ে একটা ফলাফল from functools import reduce jog = reduce(lambda a, b: a + b, numbers) print(jog) # 15
```

🗺️ কোন পথে এগাবো — Roadmap

1

🐣 Beginner (এই গাইড শেষ করলে)

Variables | Loops | Functions | Lists/Dicts | File Handling

2

🚀 Intermediate

OOP Deep | APIs (requests) | Database (SQLite) | Web Scraping | Testing

3

🧠 Advanced — যেকোনো পথ বেছে নাও

🌐 Web Dev

- Django
- Flask
- FastAPI

🤖 AI/ML

- NumPy
- Pandas
- TensorFlow

🔐 Security

- Scapy
- Impacket
- Paramiko

🎓 তুমি এখন জানো

- ⚡ List Comprehension — একলাইনে list
- λ Lambda — নামছাড়া function
- 🎁 Decorators — function-এ extra শক্তি
- ⚡ Generators — মেমরি বাঁচিয়ে iteration
- 🗺️ `map(), filter(), reduce()` — functional programming

🐍 "Code করো, ভুল করো, শেখো, আবার করো।"

"Code, make mistakes, learn, repeat."

এই গাইড সম্পূর্ণ শিক্ষামূলক উদ্দেশ্যে। প্রতিটি কোড নিজে লিখে রান করো — শুধু পড়লে শেখা হয় না।

Python Zero to Advanced Guide — v1.0 | 14 Chapters | 100+ Examples